

MAC 설계 프로젝트

조현지

chohyjee1021@gmail.com

01 Project Overview

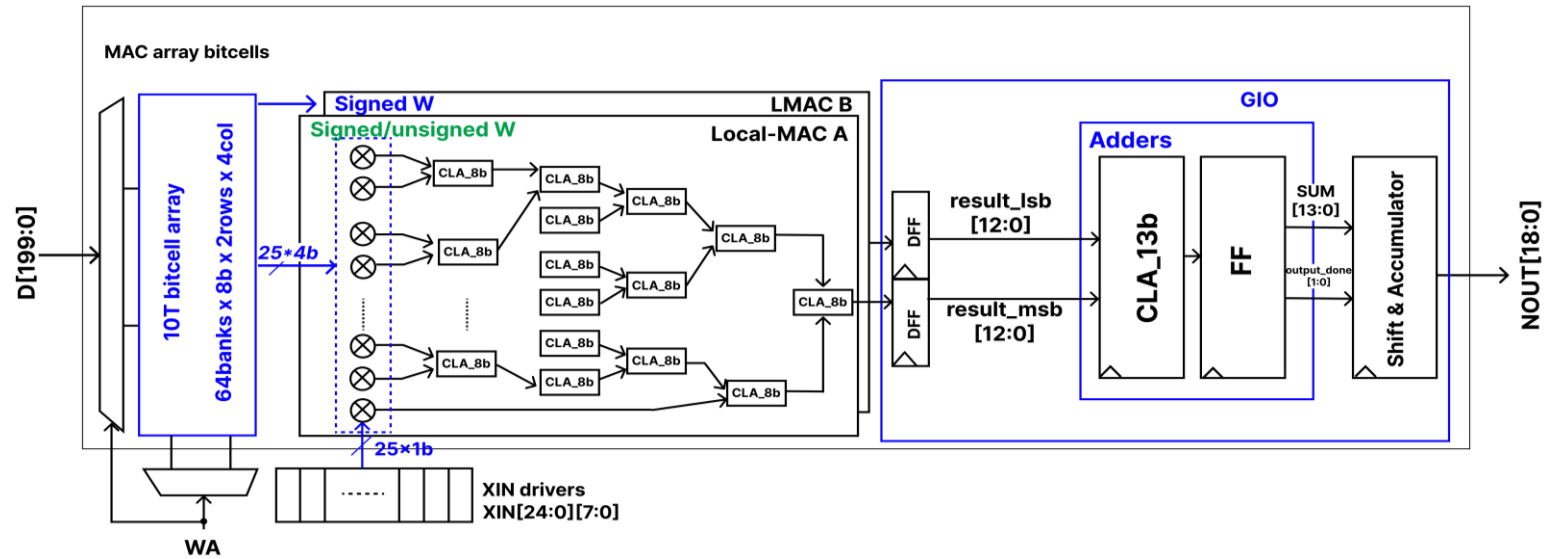
02 MAC Mechanism

03 Module Breakdown

04 FPGA

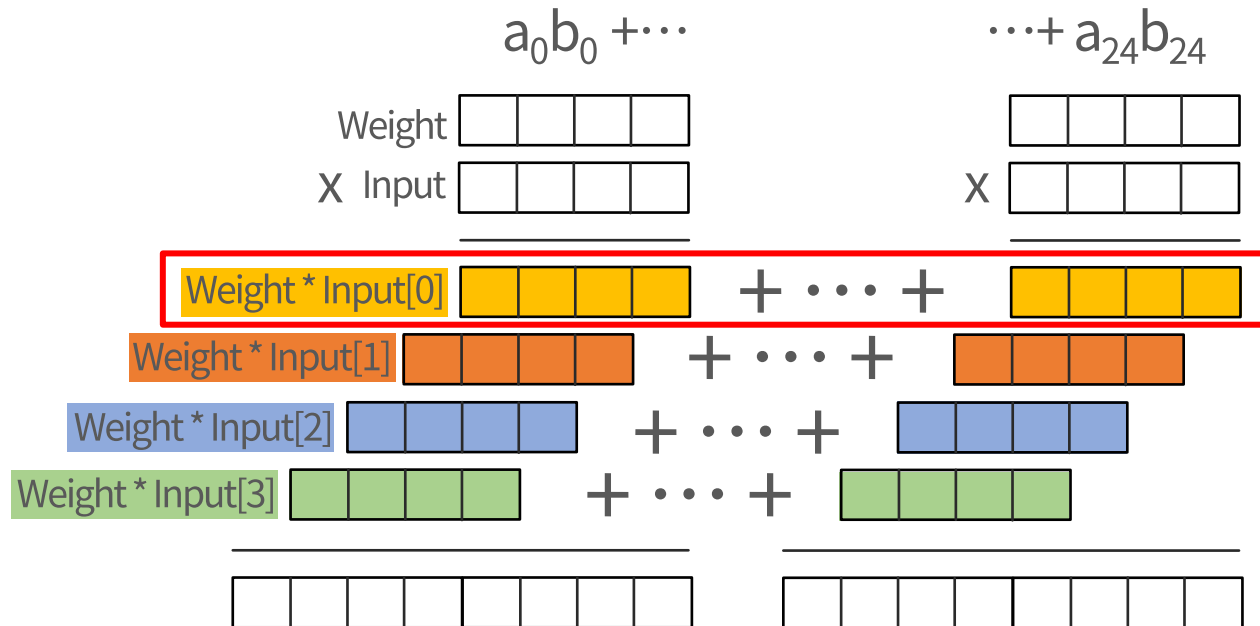
05 Results

01 Project Overview



기간	2024.03.04 ~ 2024.11.15
소속	교내 전자공학종합설계 프로젝트
사용 툴	Modelsim, Vivado, OpenROAD (언어: Verilog)
목표	Input 4b/8b * Weight 4b/8b 4가지 케이스로 25번의 MAC 연산 수행하는 구조 디자인 후 PPA 개선
맡은 역할	Verilog로 LMAC, Adders 설계, OpenROAD로 Back End
결과	초기 연산 구조에 비해 Power 소모 22프로 증가, Performance 12프로 향상, Area 21프로 증가

02 MAC Mechanism

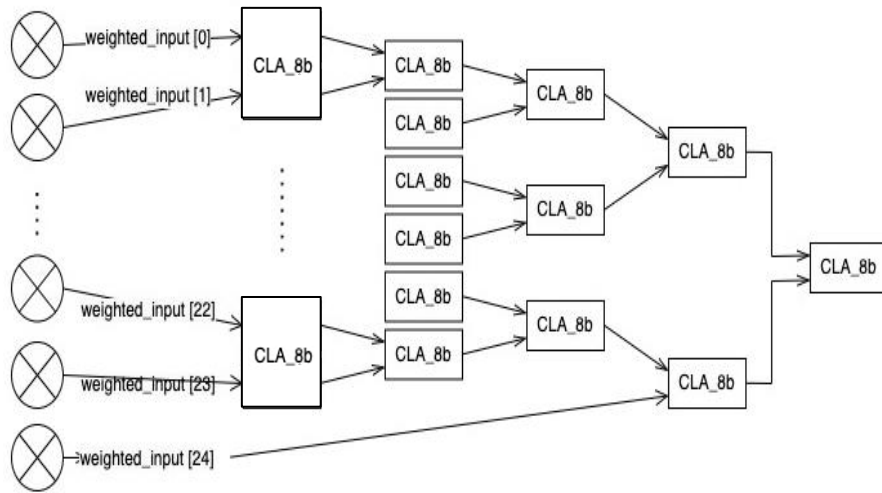


```
At time 0: reset is pressed; DCIM macro initialized!
At time 5: reset is pressed; DCIM macro initialized!
At time 95: in_width = 0, w_width = 0, final_sum = 125
At time 170: reset is pressed; DCIM macro initialized!
At time 175: reset is pressed; DCIM macro initialized!
At time 265: in_width = 0, w_width = 1, final_sum = 10125
At time 340: reset is pressed; DCIM macro initialized!
At time 345: reset is pressed; DCIM macro initialized!
At time 475: in_width = 1, w_width = 0, final_sum = 4125
At time 510: reset is pressed; DCIM macro initialized!
At time 515: reset is pressed; DCIM macro initialized!
At time 645: in_width = 1, w_width = 1, final_sum = 334125
** Note: $stop : C:/Users/hj/Desktop/DCIM/top_tb.v(166)
Time: 670 ns Iteration: 0 Instance: /top_tb
```

Input Data	Input Buffer을 통해 bit별로 Clk Cycle마다 들어옴 (4b 0101, 8b 1010_0101)
Weight	4b, 8b 형태로 연산됨 (4b 0001, 8b 0101_0001)
LMAC	Clk Cycle마다 Input을 Bit별로 받아 Weighted Input의 합을 구함
GIO	Clk Cycle마다 들어오는 Psum을 shift&acc 형태로 더해 연산값을 구함
finalshiftacc	Input이 4b일 때는 7 Clk Cycle 이후에, 8b 일 때는 11 Clk Cyle 이후에 나옴

03 Module Breakdown

LMAC



Input	Input Data는 Lsb부터 Clk Cycle마다 1bit씩, Weight는 4bit 전체가 들어옴
Output	Adder Tree 구조를 거친 9bit Output 출력
구성	FA, LAU_4b, CLA_4b, LAU_2b, CLA_8b, LMAC
기능	25개의 Input Data 1b * Weight를 Adder Tree 구조로 CLA를 통해 더해줌

Design

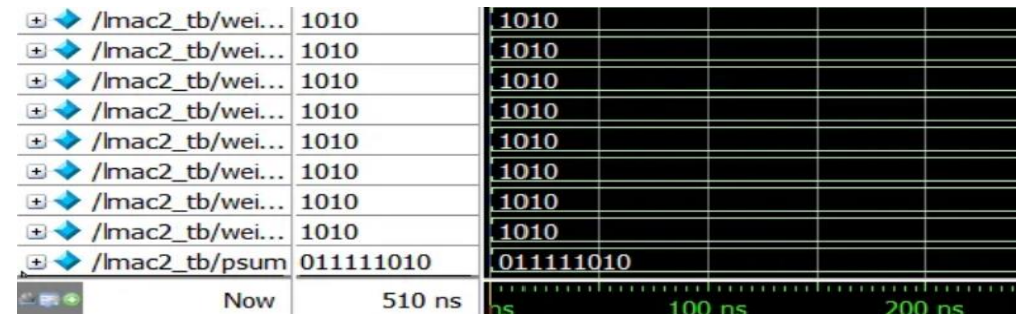
```
// add_6b_to_7b
wire [7:0] psum8b_7 [0:2];
generate
    genvar gl;
    for (gl = 0; gl < 6; gl = gl + 2) begin : add_6b_to_7b
        CLA_8b CLA8b_6(gl+1), .A(psum8b_6[gl]), .B(psum8b_6[gl+1]), .Sum(psum8b_7[gl/2]), .Cout(), .PG(), .GG());
    end
endgenerate

// add_7b_to_8b
wire [7:0] psum8b [0:1];
CLA_8b CLA8b_0(.Cin(1'b0), .A(psum8b_7[0]), .B(psum8b_7[1]), .Sum(psum8b[0]), .Cout(), .PG(), .GG());
CLA_8b CLA8b_1(.Cin(1'b0), .A(psum8b_7[2]), .B(psum8b_7[3]), .Sum(psum8b[1]), .Cout(), .PG(), .GG());

// add_8b_to_9b
wire [8:0] psum_temp;
CLA_8b CLA8b_2(.Cin(1'b0), .A(psum8b[0]), .B(psum8b[1]), .Sum(psum_temp[7:0]), .Cout(psum_temp[8]), .PG(), .GG());

assign psum = psum_temp;
endmodule
```

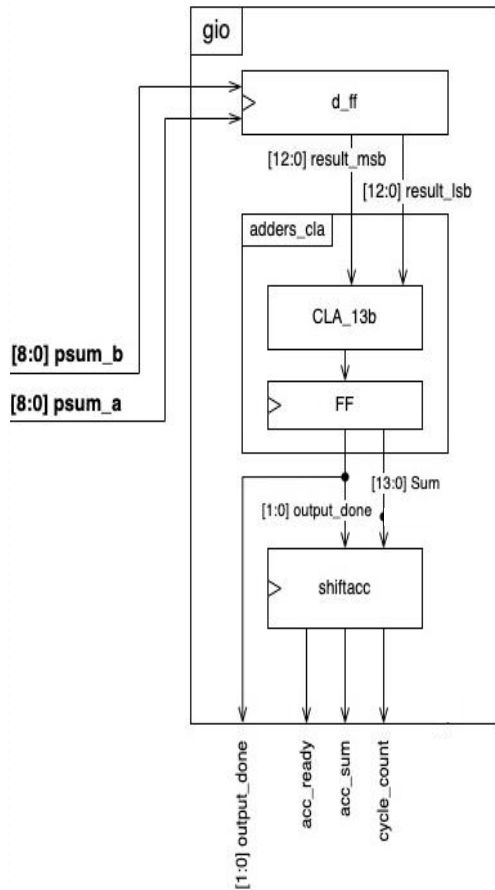
Code



Testbench

03 Module Breakdown

GIO



Input	psum_msb, psum_lsb
Output	Input 4b* Weight 4b/8b 결과
구성	dff, adders_cla, shiftacc
기능	dff: LMAC Output 자릿수 보정 adders_cla: 4b Input * Weight 의 4b/8b의 부분합 출력 Shiftacc: FSM 기반으로 각 clk cycle마다 들어오는 값을 자릿수만 큼 shift하고 acc해 더함

Design

```

module gio
(
    clk, rst_n, psum_msb, psum_lsb, in_width, w_width,
    acc_sum, output_done, acc_ready
);

input clk, rst_n, in_width, w_width;
input [8:0] psum_msb, psum_lsb;

output [18:0] acc_sum;
output [1:0] output_done;
output acc_ready;

wire [13:0] output_sum;

wire [12:0] result_lsb;
wire [12:0] result_msb;
wire [1:0] output_done;

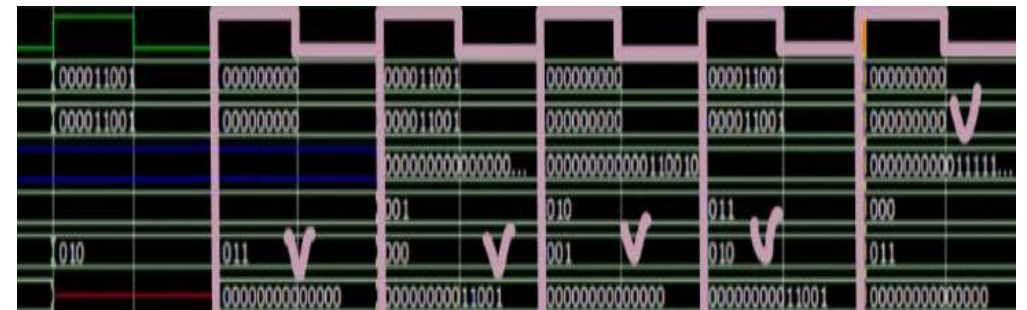
d_ff d_ff_0(clk(clk), .rst_n(rst_n), .w_width(w_width), .psum_msb(psum_msb), .psum_lsb(psum_lsb), .result_msb(result_msb), .result_lsb(result_lsb));

adders_cla CLA(clk(clk), .rst_n(rst_n), .Cin(1'b0), .A(result_msb), .B(result_lsb), .Sum(output_sum), .output_done(output_done));

shiftacc shiftacc_0(in_width(in_width), .clk(clk), .rst_n(rst_n), .psum(output_sum), .output_done(output_done), .acc_sum(acc_sum), .acc_ready(acc_ready));
endmodule

```

Code



Testbench

03 Module Breakdown

finalshiftacc

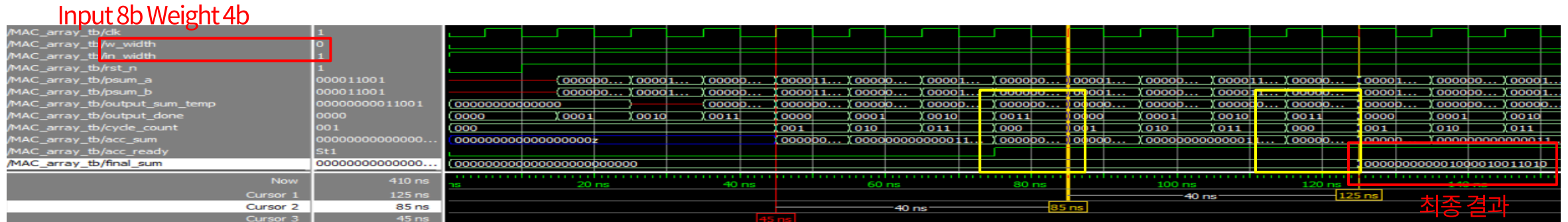
Input	Input 4b * Weight 4b/8b 최종 결과
Output	Input 4b/8b * Weight 4b/8b 최종 결과
구성	finalshiftacc
기능	Input이 8b일 때 adders_cla의 출력값을 저장, 7 clk cycle 이후에 자릿수 보정된 Input Msb* Weight와 더해 최종 결과 출력

```
input clk, acc_ready, in_width, rst_n;
input [1:0] output_done;
input [18:0] acc_sum;
output reg [23:0] final_sum;
reg [18:0] temp_sum;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        final_sum <= 0;
        temp_sum <= 0;
    end
    else begin
        if (output_done == 3 && acc_ready) begin
            if (in_width == 0) begin
                final_sum <= acc_sum;
            end
            else if (in_width == 1) begin
                if (temp_sum == 0) begin
                    temp_sum <= acc_sum;
                end
            end
            else begin
                final_sum <= temp_sum + {1'b0, acc_sum, 4'b0000};
                temp_sum <= 0;
            end
        end
    end
end
```

Design

Code



Testbench

04 FPGA Simulation

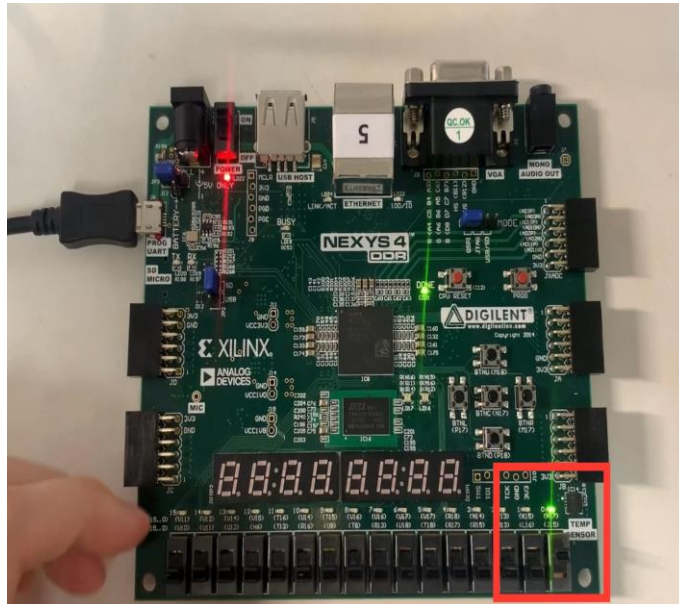
Code

```
3'b001: begin
  if (final_sum == 24'd4250) begin // input 4b x weight 8b, input 8b x weight 4b
    current_state <= green;
  end else if (final_sum == 24'd250) begin // input 4b x weight 4b
    current_state <= red;
  end else if (final_sum == 24'd72250) begin // input 8b x weight 8b
    current_state <= blue;
  end
end
green: begin
  LED16_G <= 1'b1;
end
red: begin
  LED16_R <= 1'b1;
end
blue: begin
  LED16_B <= 1'b1;
end
endcase
```

Code Mechanism

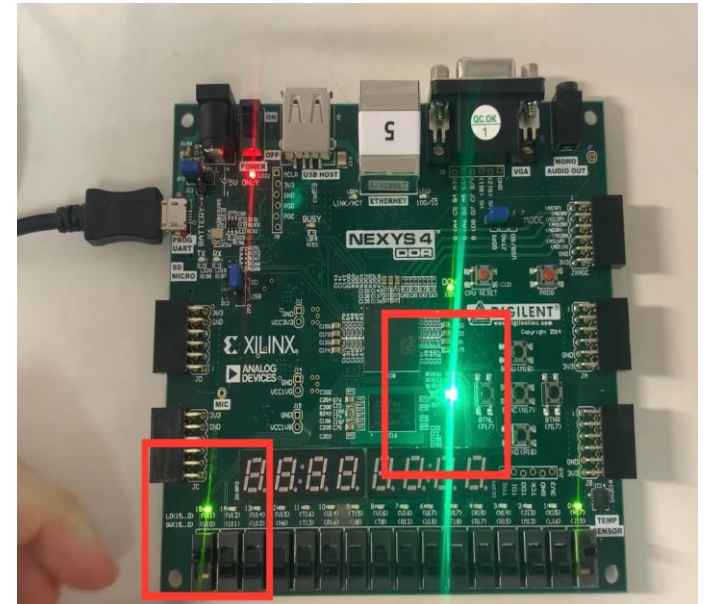
- 기존 Top Module에 Display module만 추가해 input/weight의 bit수를 사용자가 설정할 수 있게 함.
- 연산된 output의 값이 예상값과 맞는지 비교. 일치시 led가 켜짐.

Simulation



Input

- 우측에 위치하는 스위치 두 개로 Input/weight 크기를 사용자가 지정.
- 스위치를 올렸을 때는 8b, 올리지 않았을 때는 4b로 설정됨.
- 사진에서 input은 8b, weight는 4b

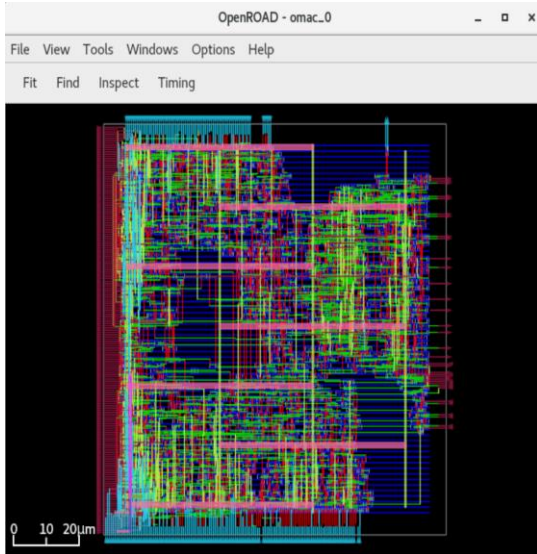


Output

- 좌측에 위치하는 스위치를 올리면 그 때의 Input/weight 스위치 상태대로 bit가 지정됨
- Input이 8b, weight가 4b일 때에 연산된 값이 예상값과 동일해 초록색 led가 켜지는 것을 확인

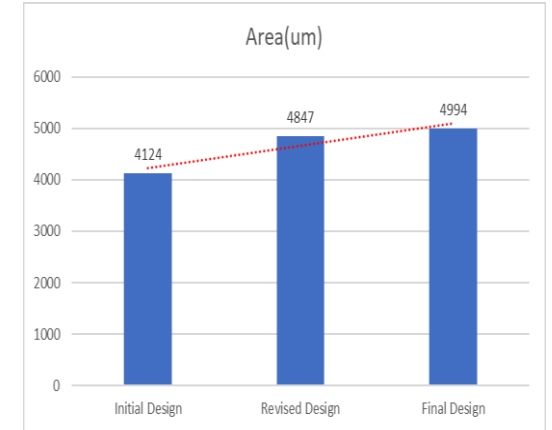
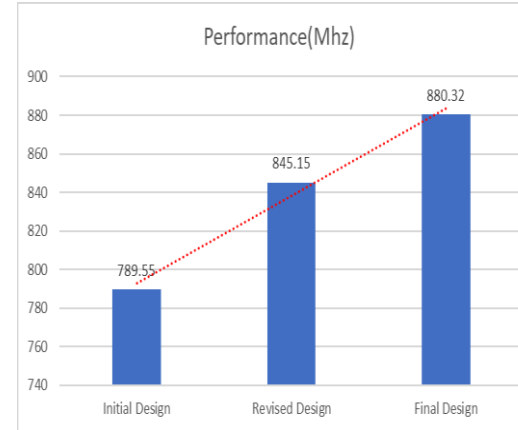
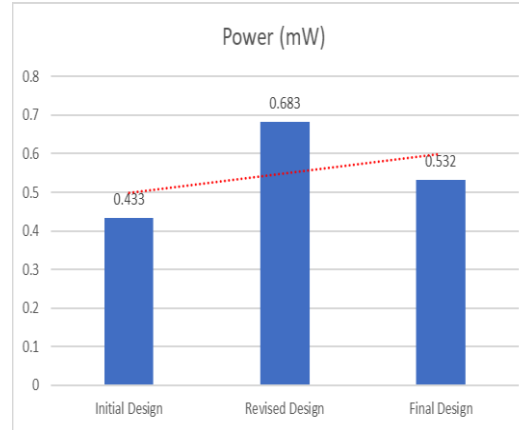
04 Results

OpenROAD



Clk Period	10ns
Core Utilization	50
Place Density	0.62
Core Aspect Ratio	1
Core Margin	5

PPA



Initial Design	Verilog 연산자만을 사용한 기본 구조
Revised Design	· 연산 부분을 CLA로 대체 · 8b CLA를 FA 8개, Lookahead Unit 1개로 구성 · 13b CLA를 FA 13개, Lookahead Unit 1개로 구성
Final Design	· 기존 Revised Design에서 CLA를 4b 기준으로 나눠 8b CLA는 4b CLA 2개와 LAU 1개 · 13b CLA는 4b CLA 3개, FA 1개, LAU 1개로 구성

Power 소모 22프로 증가, Performance 12프로 향상, Area 21프로 증가